



Unity를 이용한 2D 게임프로그래밍

Lecture 1

Unity 사용해 보기

강영민

동명대학교 게임학부

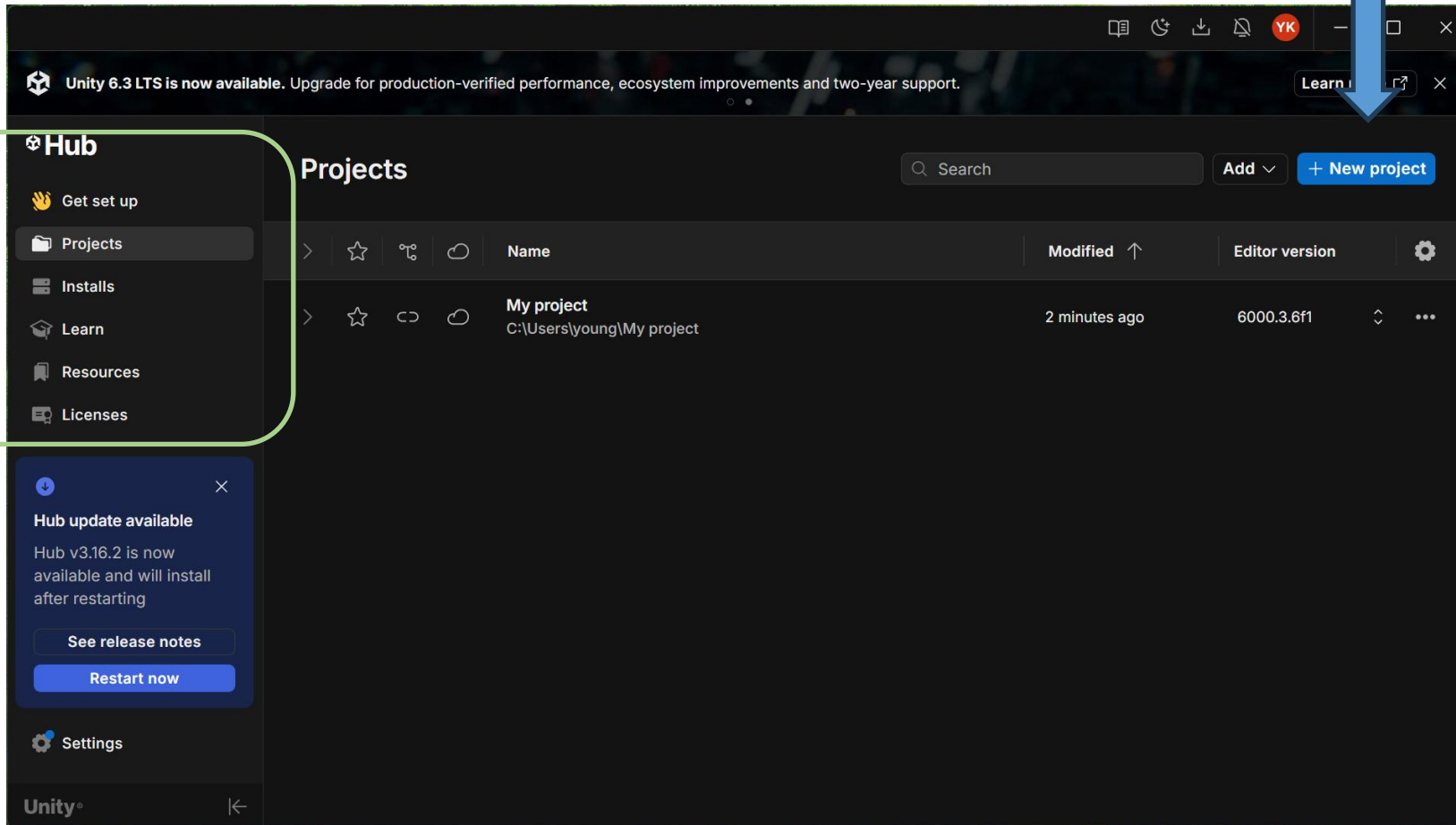
학습목표

- 유니티에 익숙해 진다.
- 간단한 프로젝트를 만들어 본다.
- 프로젝트에 C# 코드를 추가해 본다.
- 2D 게임과 3D 게임의 차이에 대해 생각해 본다.

설치한 Unity Hub를 실행해 보자

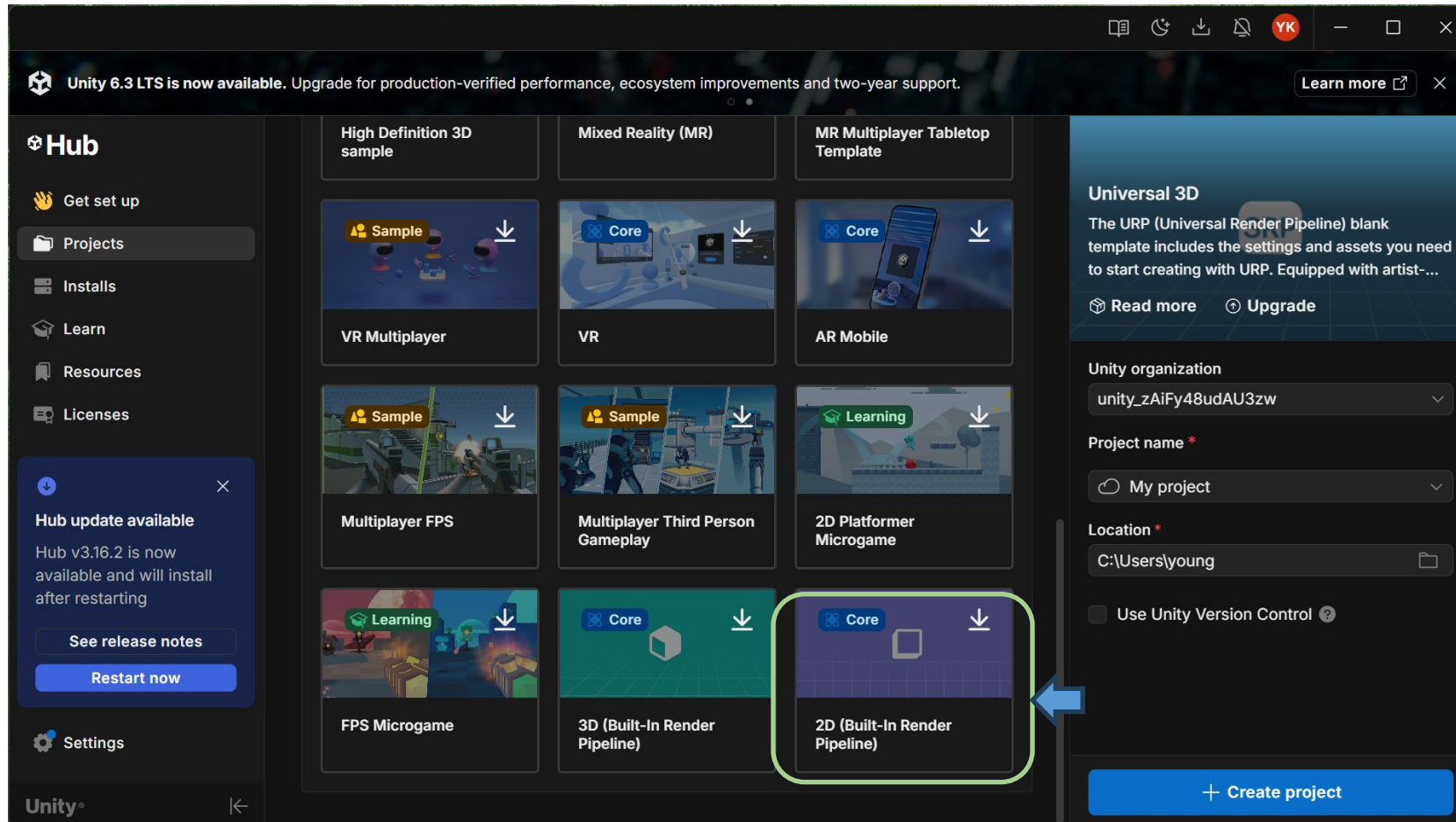
- 좌측 탭 메뉴로 실행해 보자

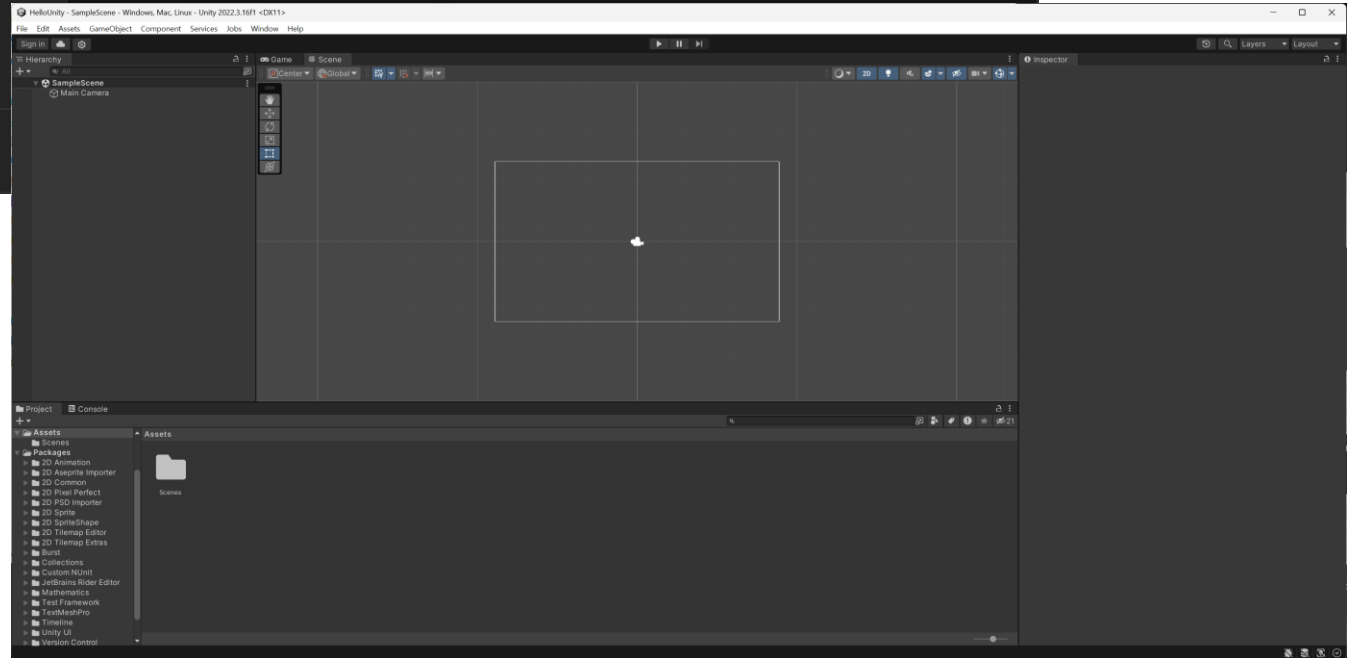
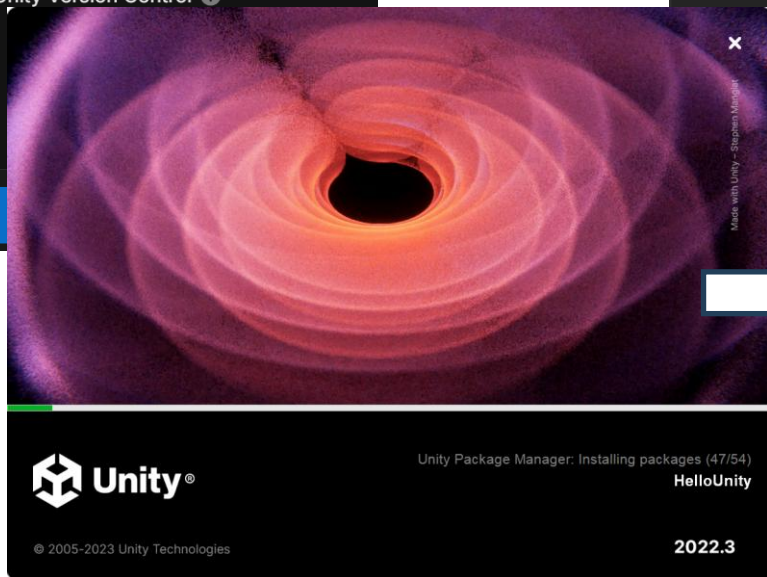
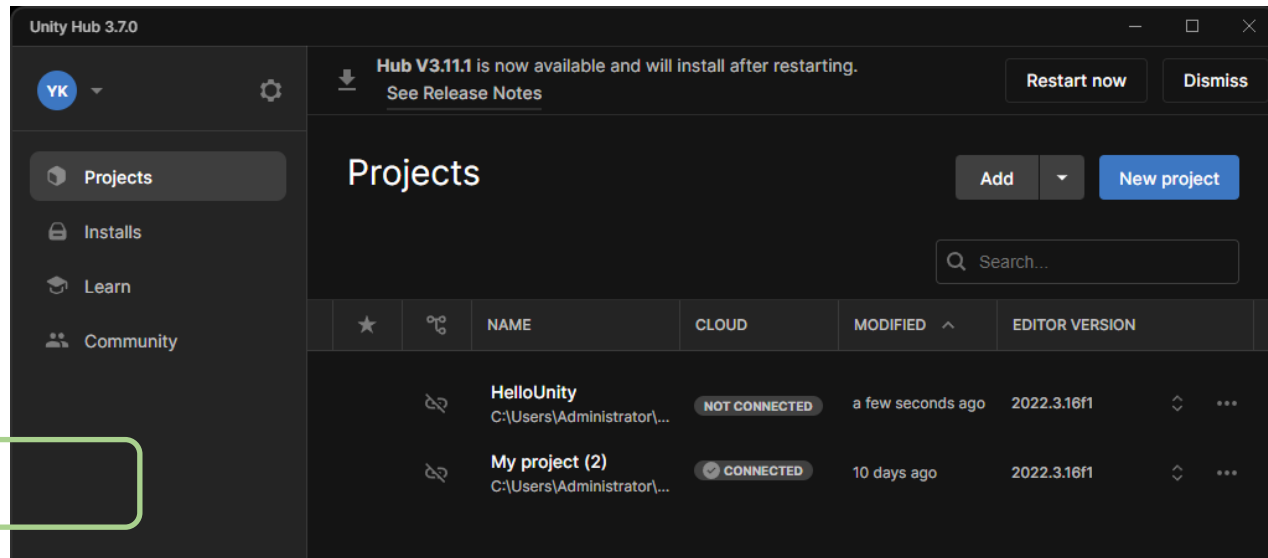
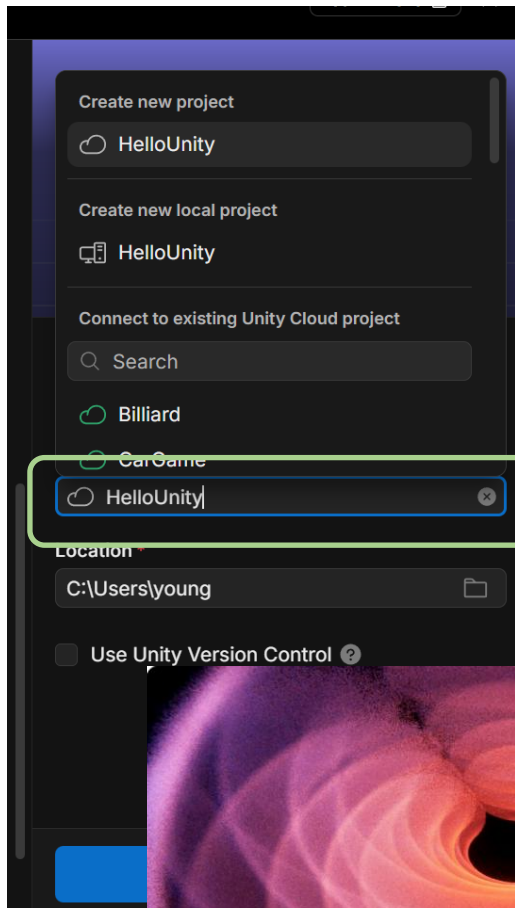
새로운 프로젝트 생성



2D 게임프로젝트를 생성해 보자

- 2D (Built-in Render Pipeline) 선택

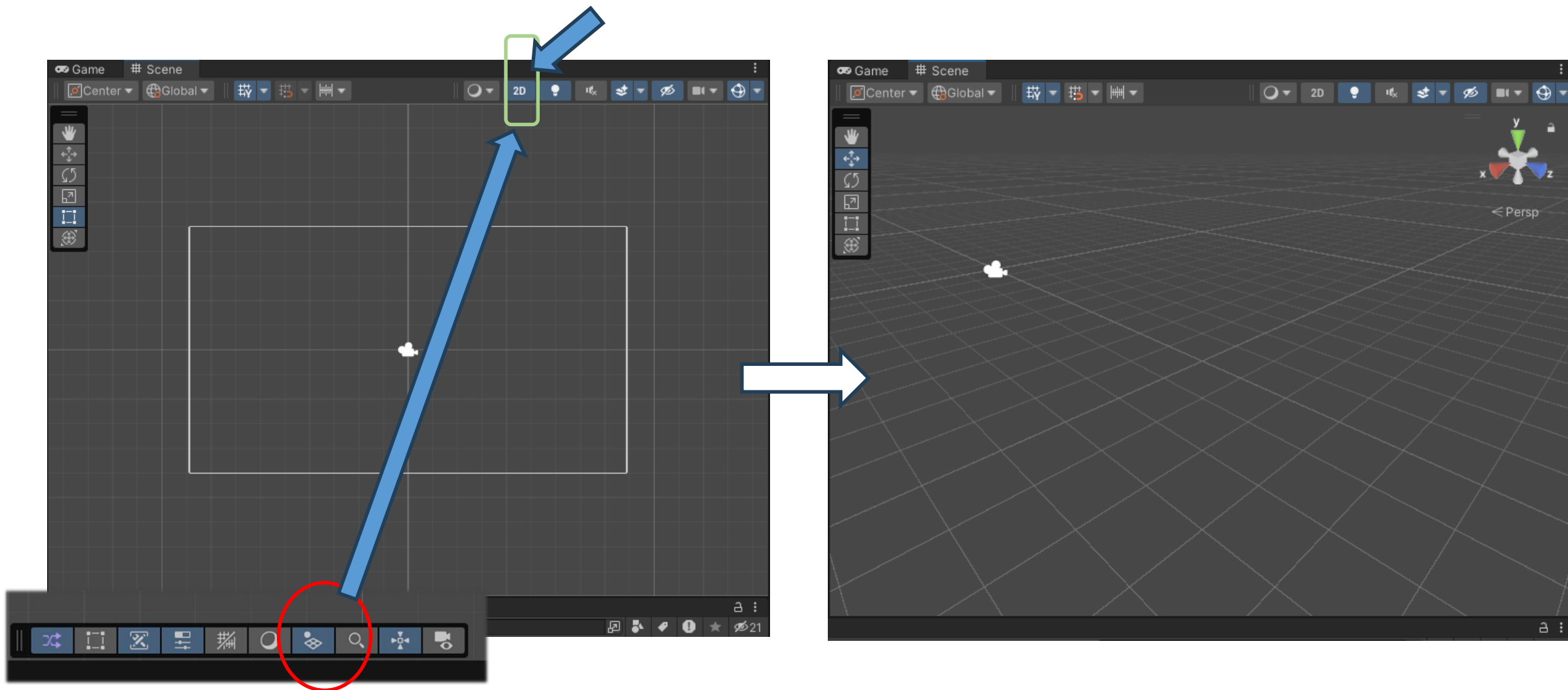




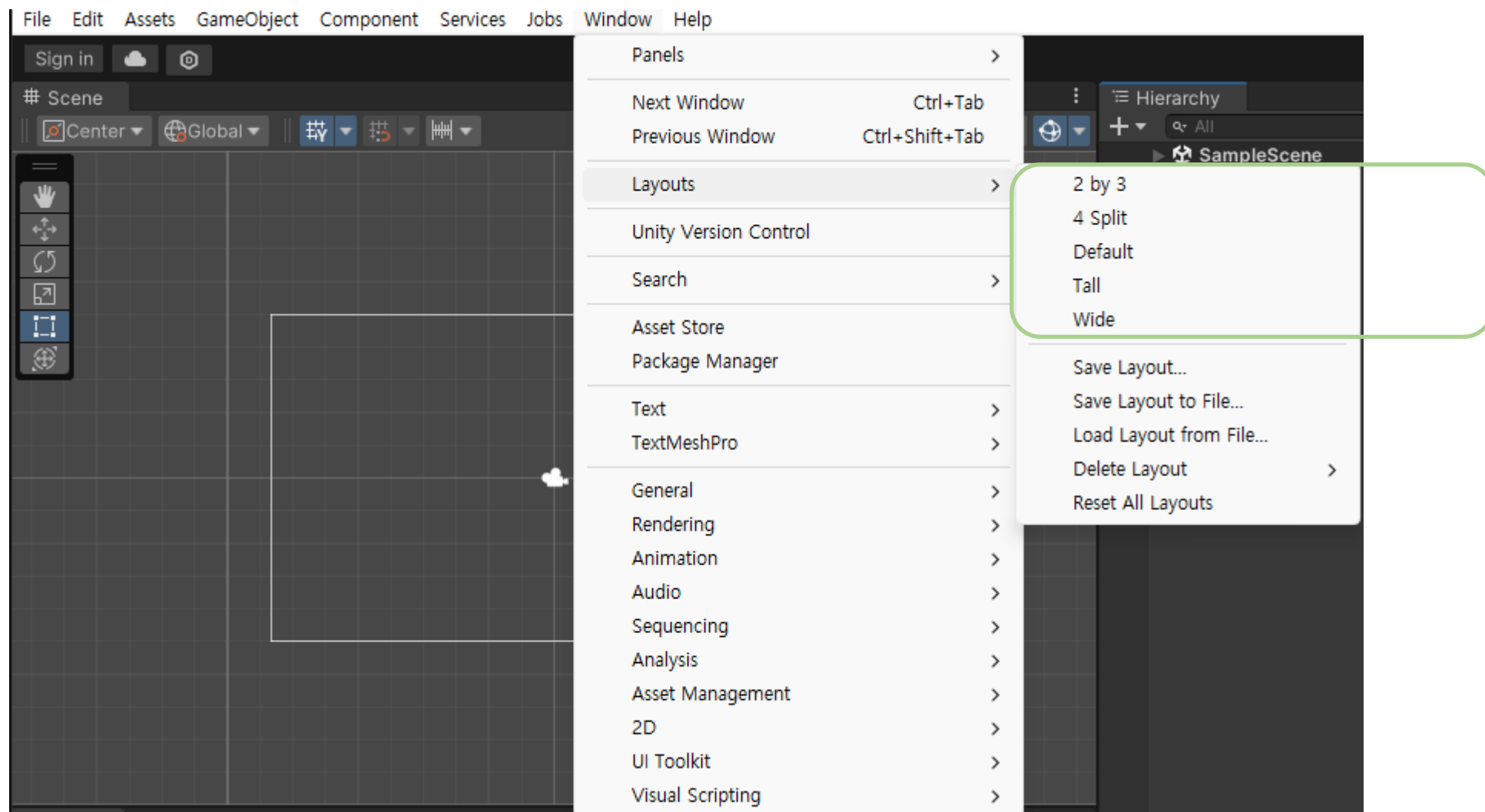
유니티 엔진 실행됨

유니티 엔진 실행되고 프로젝트가 로딩된 상태

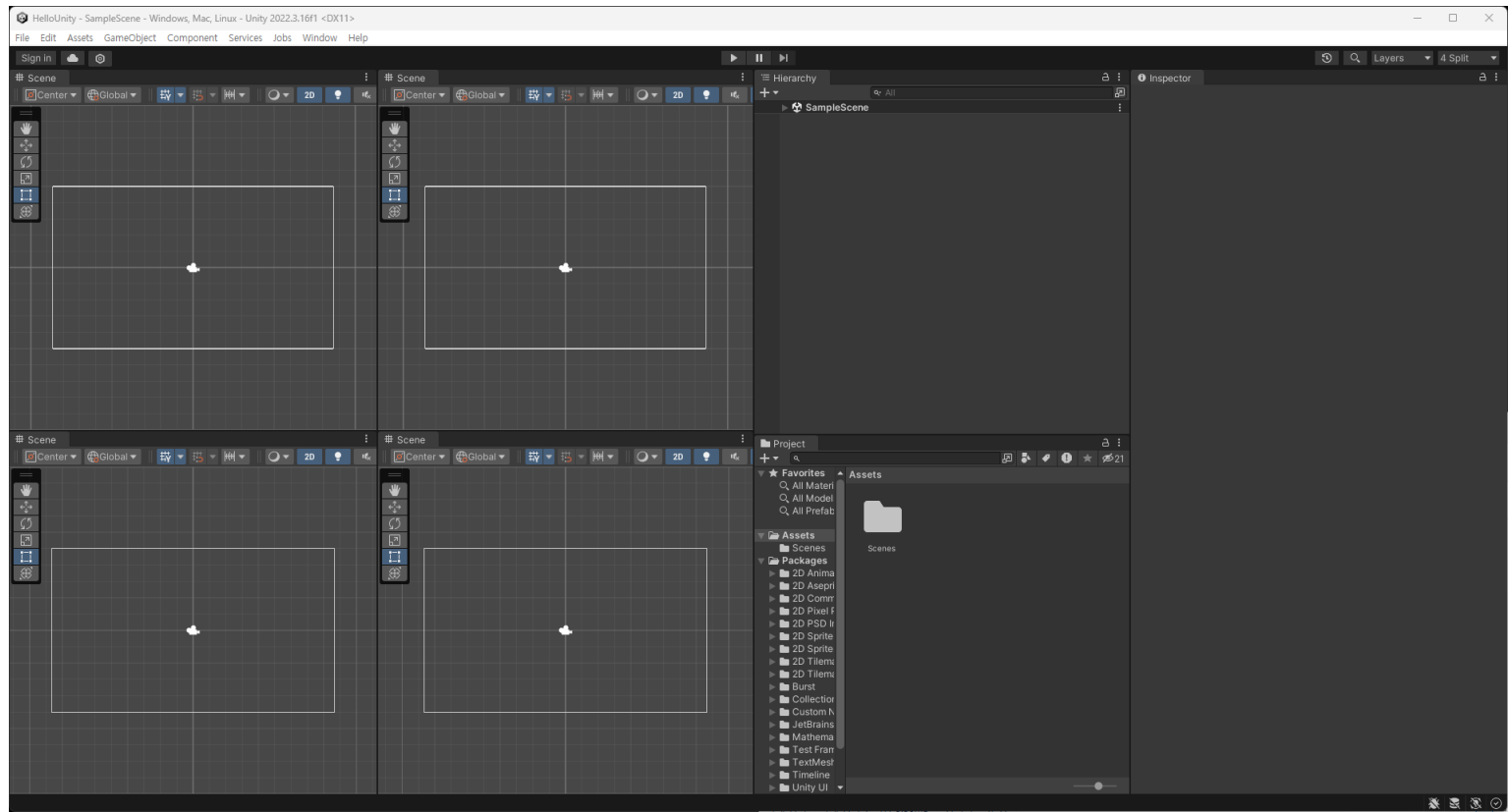
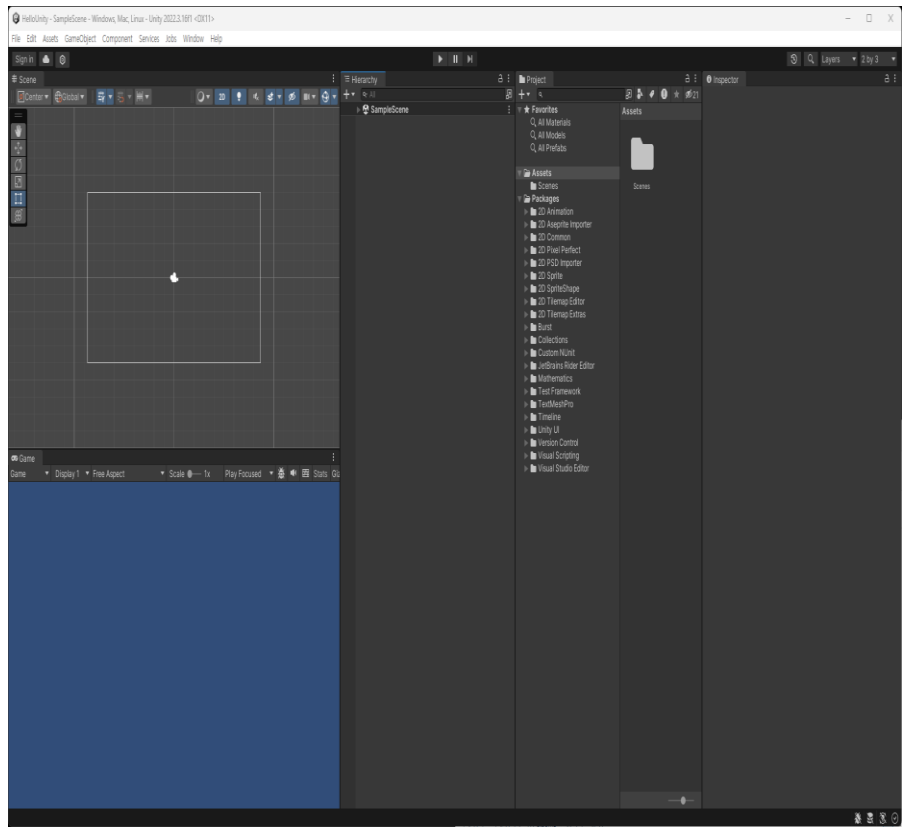
- 2D 공간은 3D 공간의 일부



다양한 윈도우 레이아웃 테스트

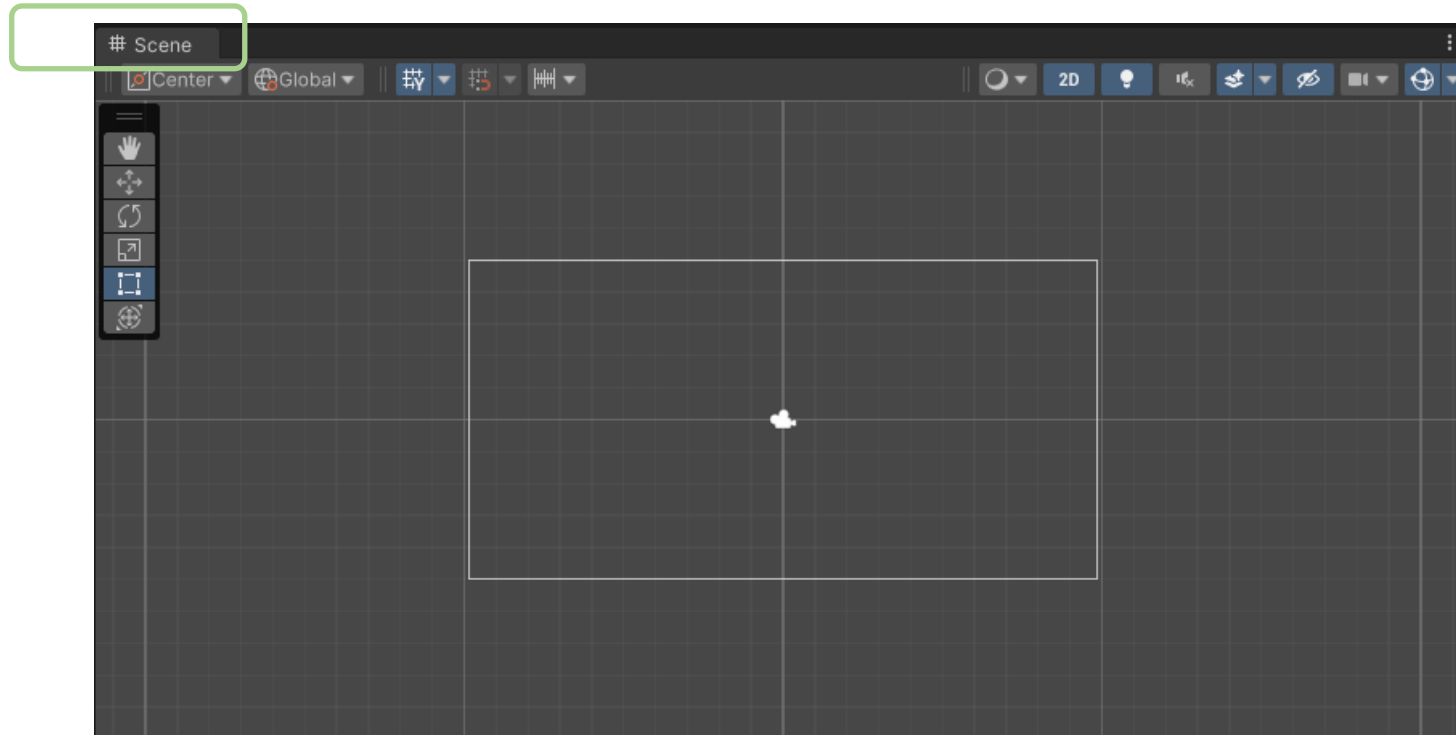


다양한 윈도 레이아웃 테스트



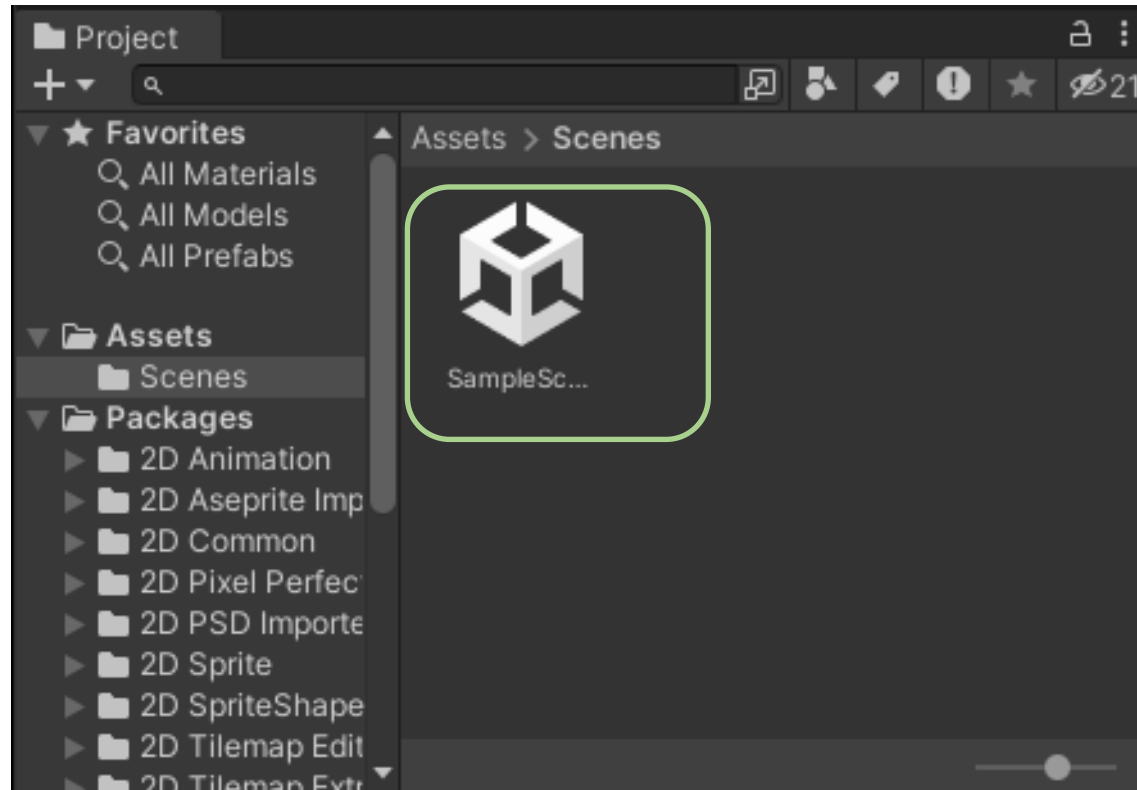
장면(Scene) – 게임 레벨

- 장면은 유니티에서 사용되는 일종의 Asset
- 장면에는 유니티 게임 프로젝트의 전부 혹은 일부 콘텐츠가 담김
- 유니티 프로젝트에서 개발은 이 장면 단위로 이루어짐



기본 장면 – SampleScene

- 유니티 프로젝트를 생성하면 SampleScene이 생성됨

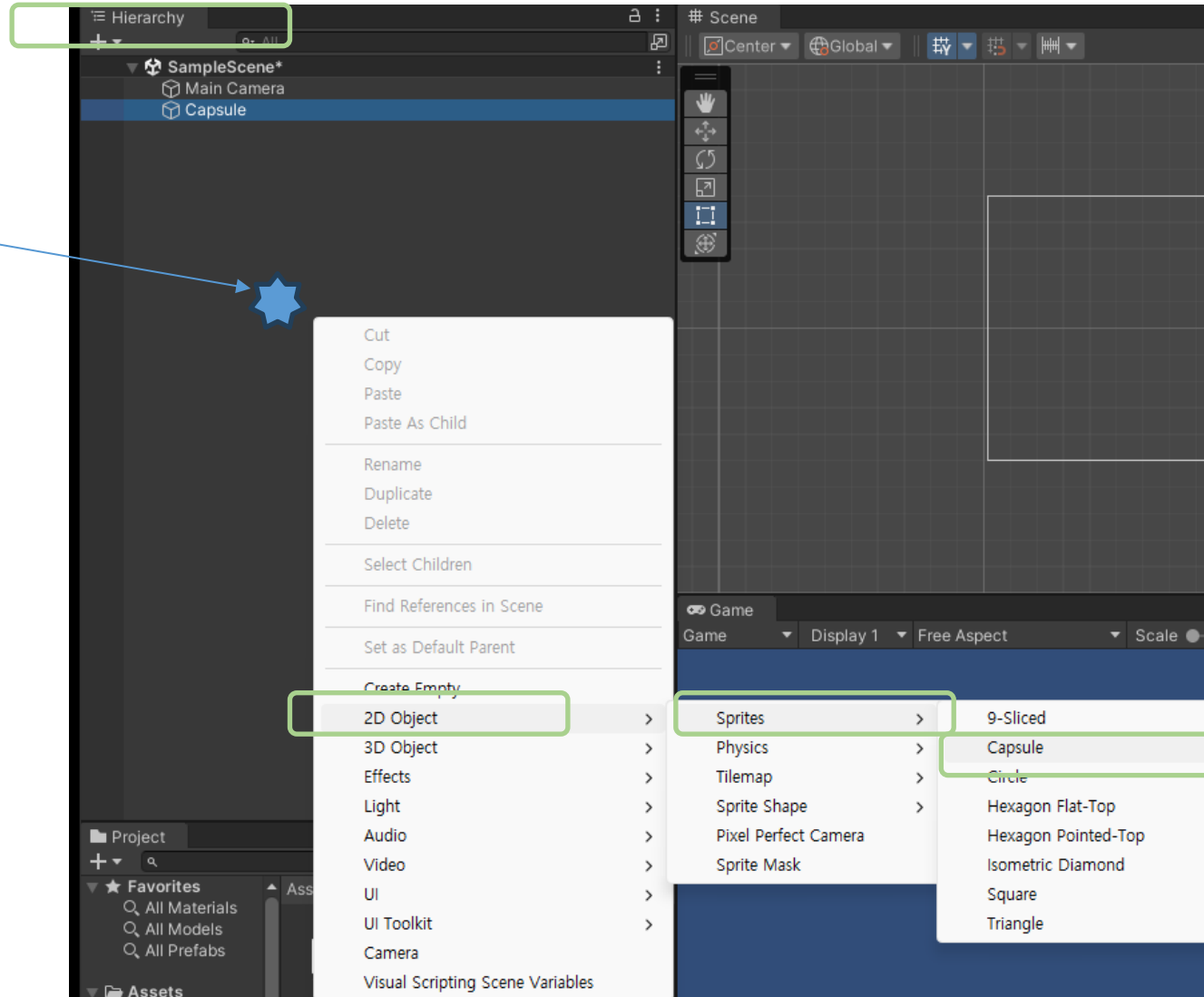


간단한 객체를 넣고 움직여 보자

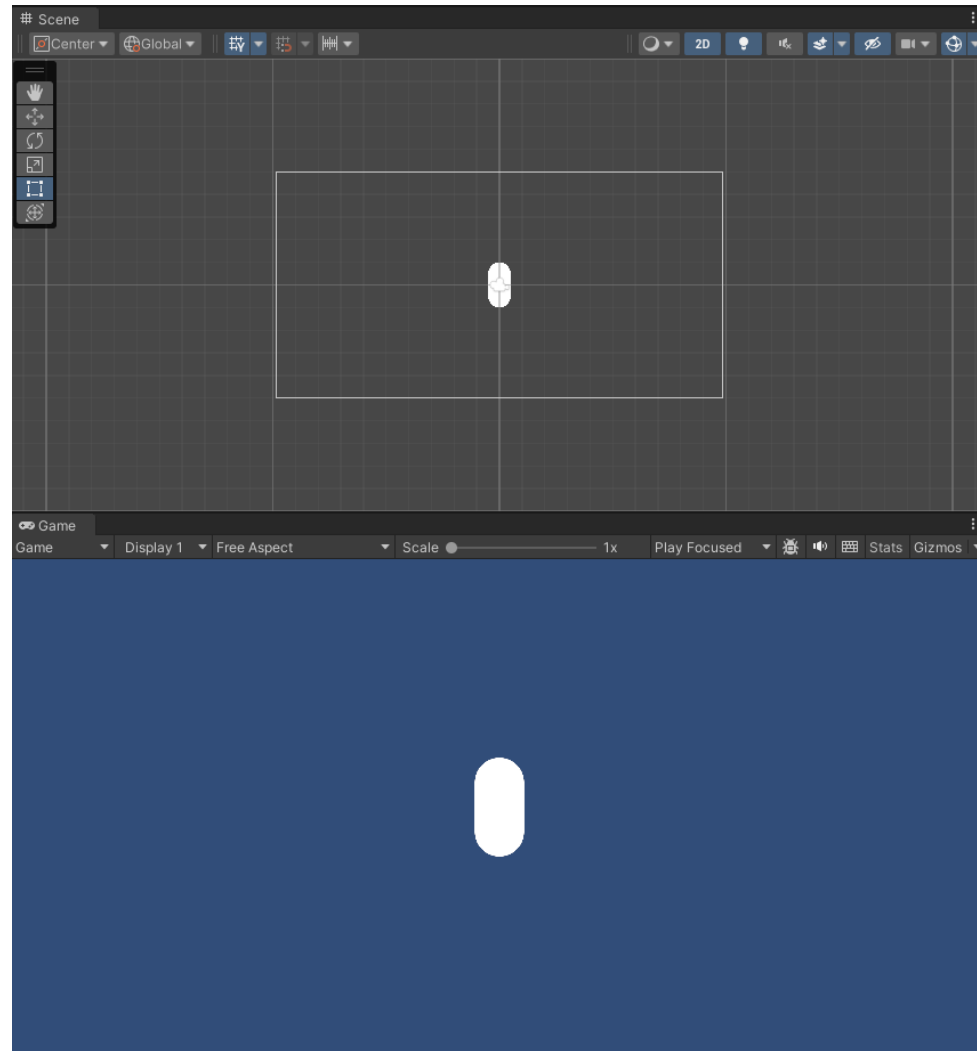
- 2D 게임 → 스프라이트 객체 사용
 - 캡슐 스프라이트 추가
 - 캡슐 스프라이트에 C# 코드 연결
 - 코딩을 통해 스프라이트 움직이기

캡슐 스프라이트 추가

Right click!



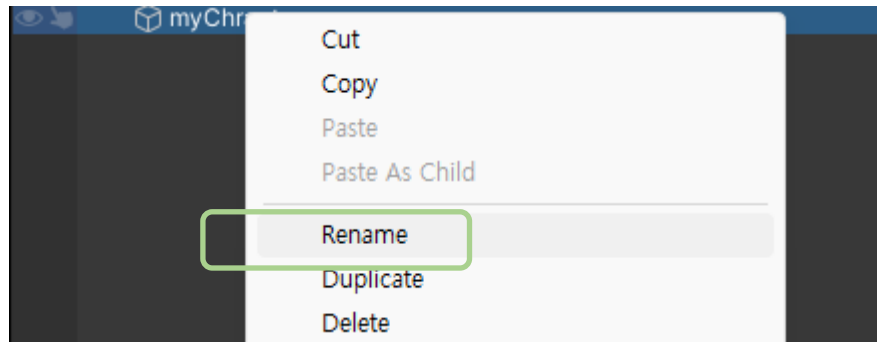
캡슐 스프라이트 확인



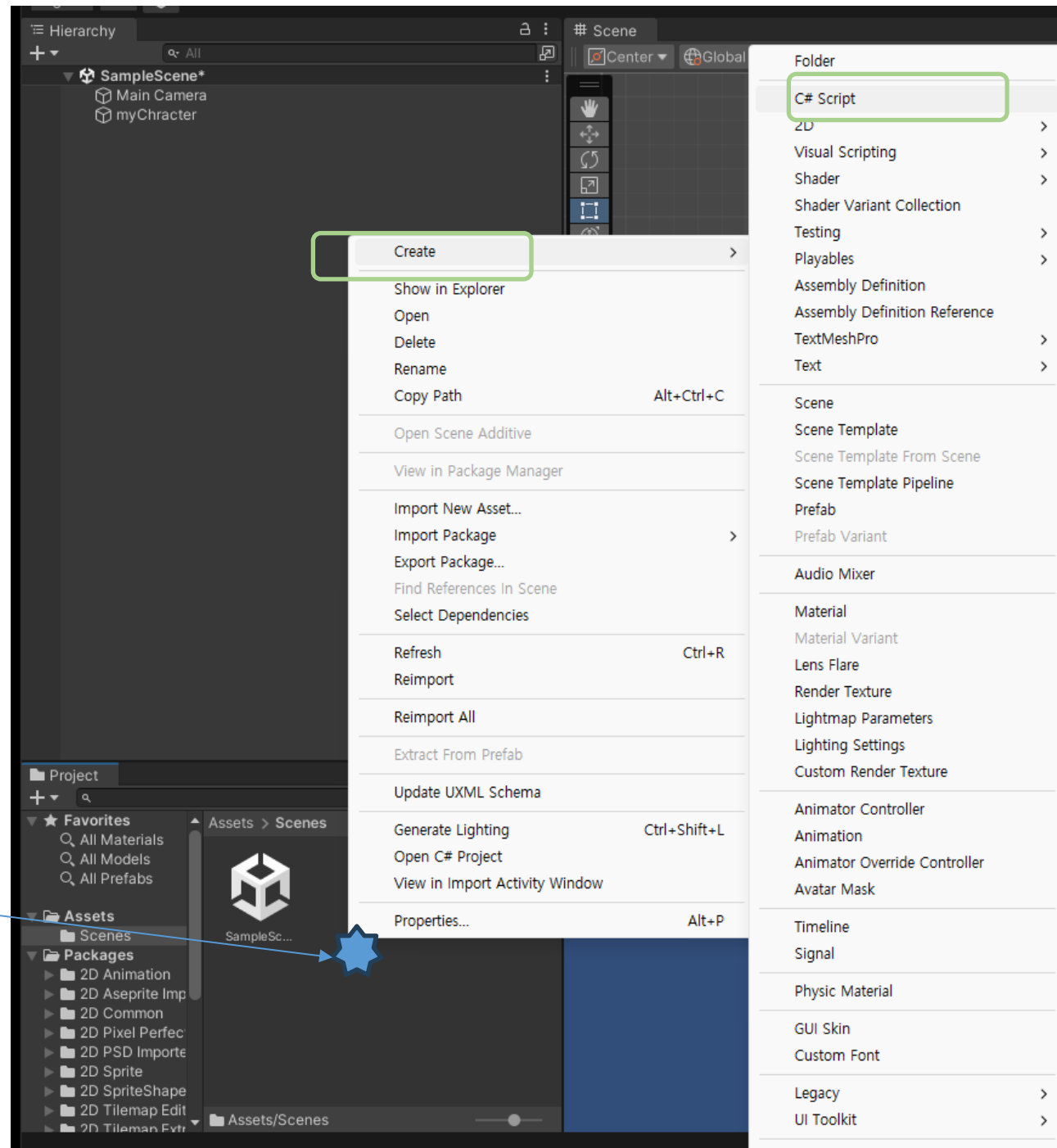
캡슐 스프라이트 움직이기 위한 “method” 사용

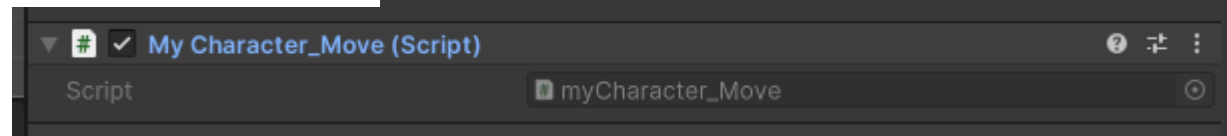
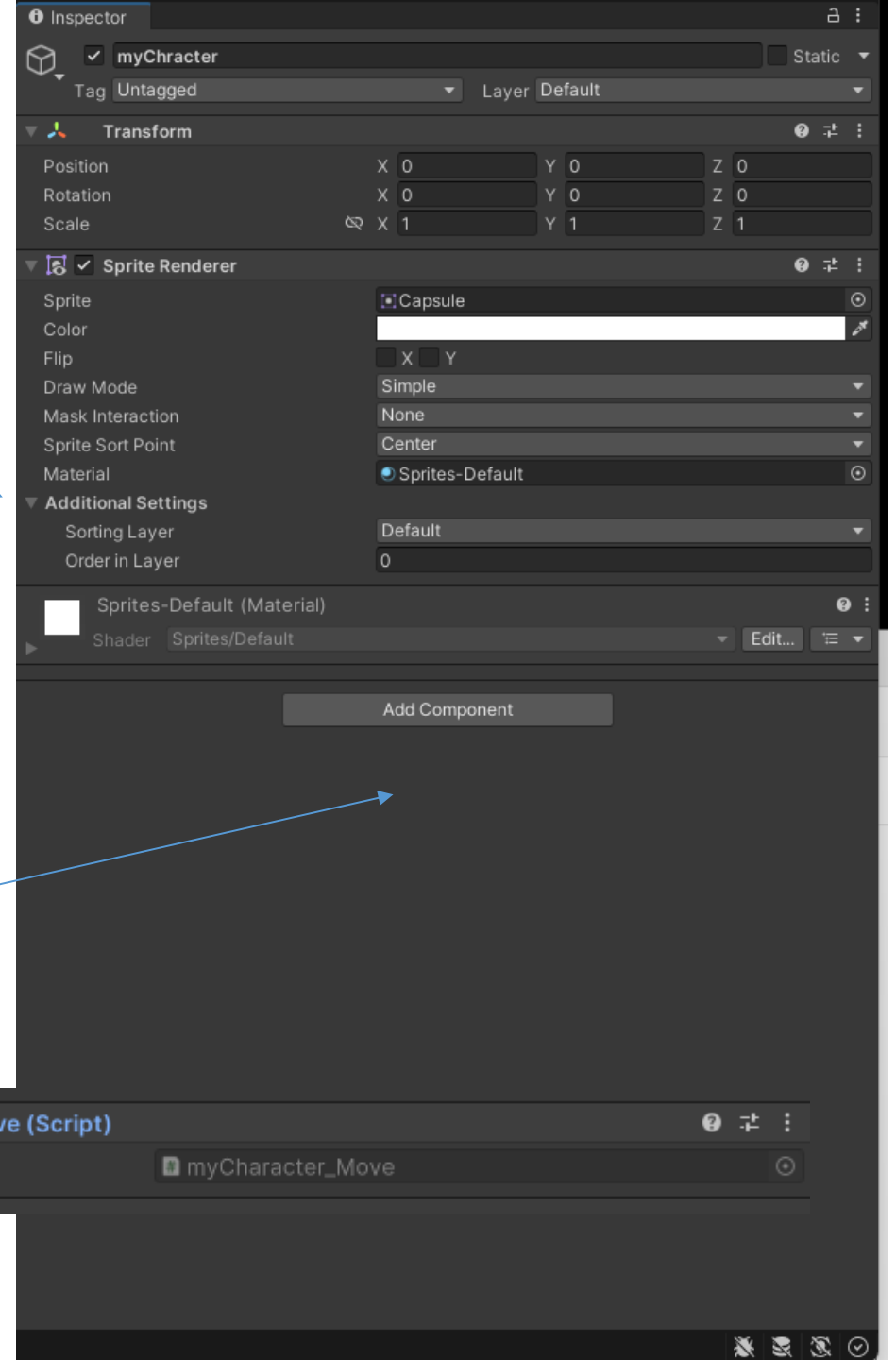
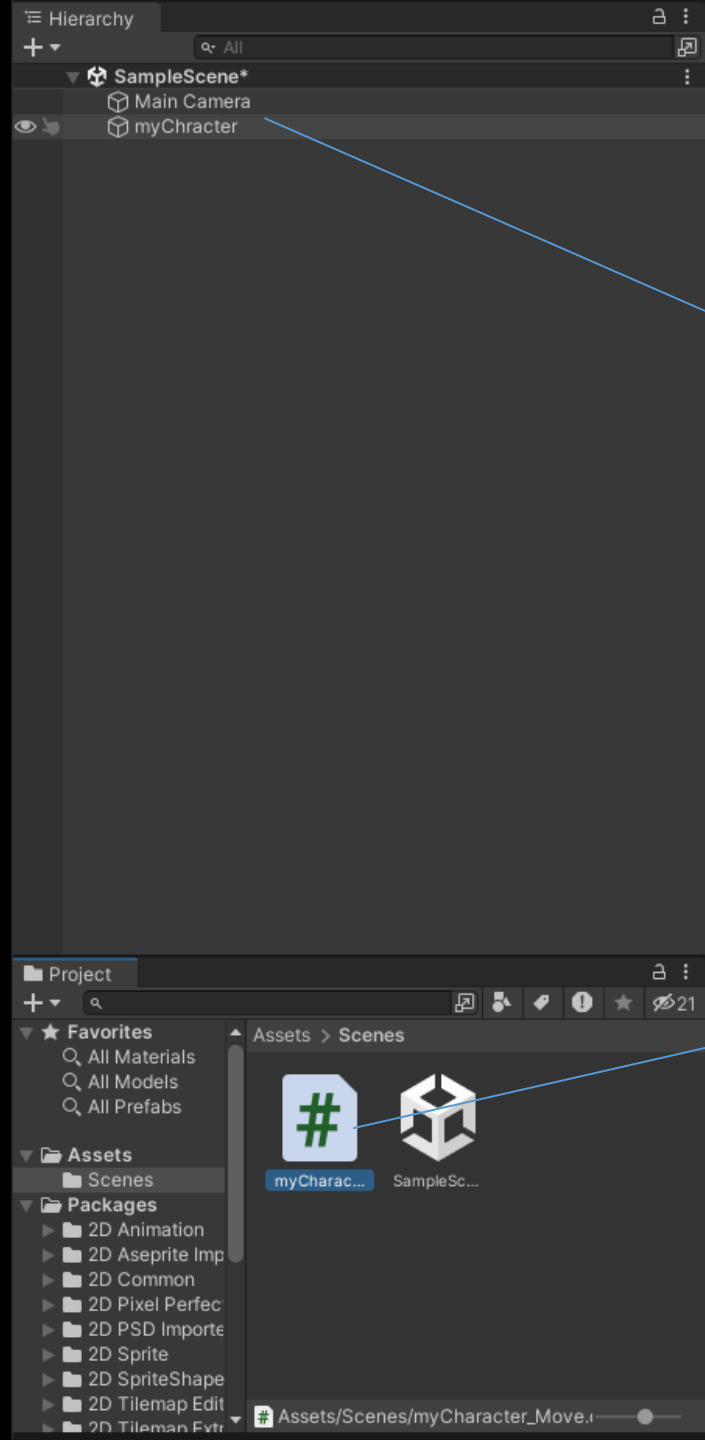
- 유니티의 메소드
 - 우리의 게임이 무엇인가를 할 수 있도록 하는 코드의 덩어리를 실행시키는 도구
 - 함수와 비슷한 개념 - 메소드는 객체에 딸린 함수
- 사용가능한 메소드
 - 유니티에서 제공하는 메소드
 - 우리가 직접 만든 메소드
- 메소드를 만들고 실행하는 것
 - 만들기: 무엇인가를 할 수 있는 코드를 메소드로 작성해 두는 것
 - 실행: 만들어 놓은 메소드를 호출하여 해당 동작이 이루어지게 하는 것

스크립트를 연결하기

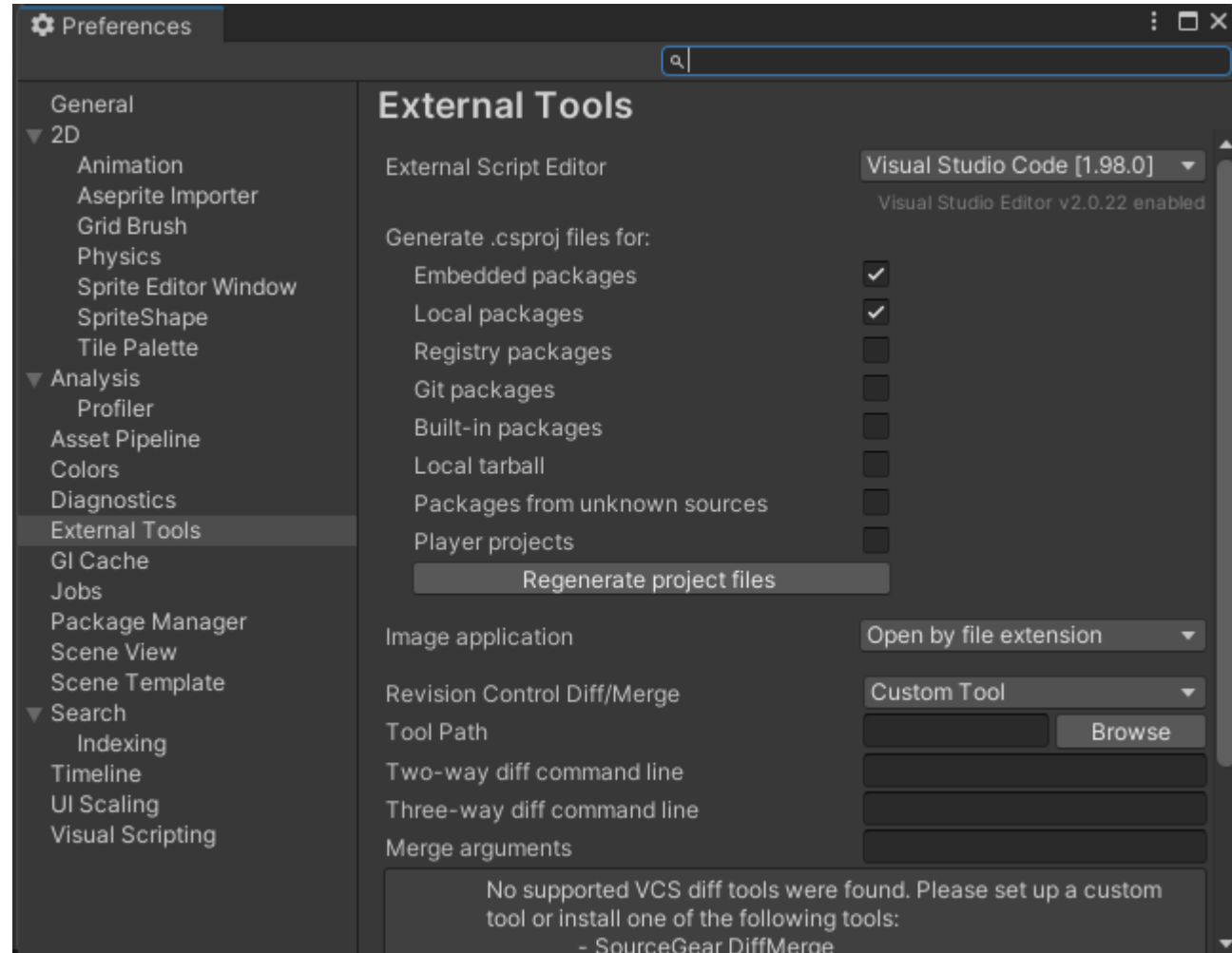


Right click!

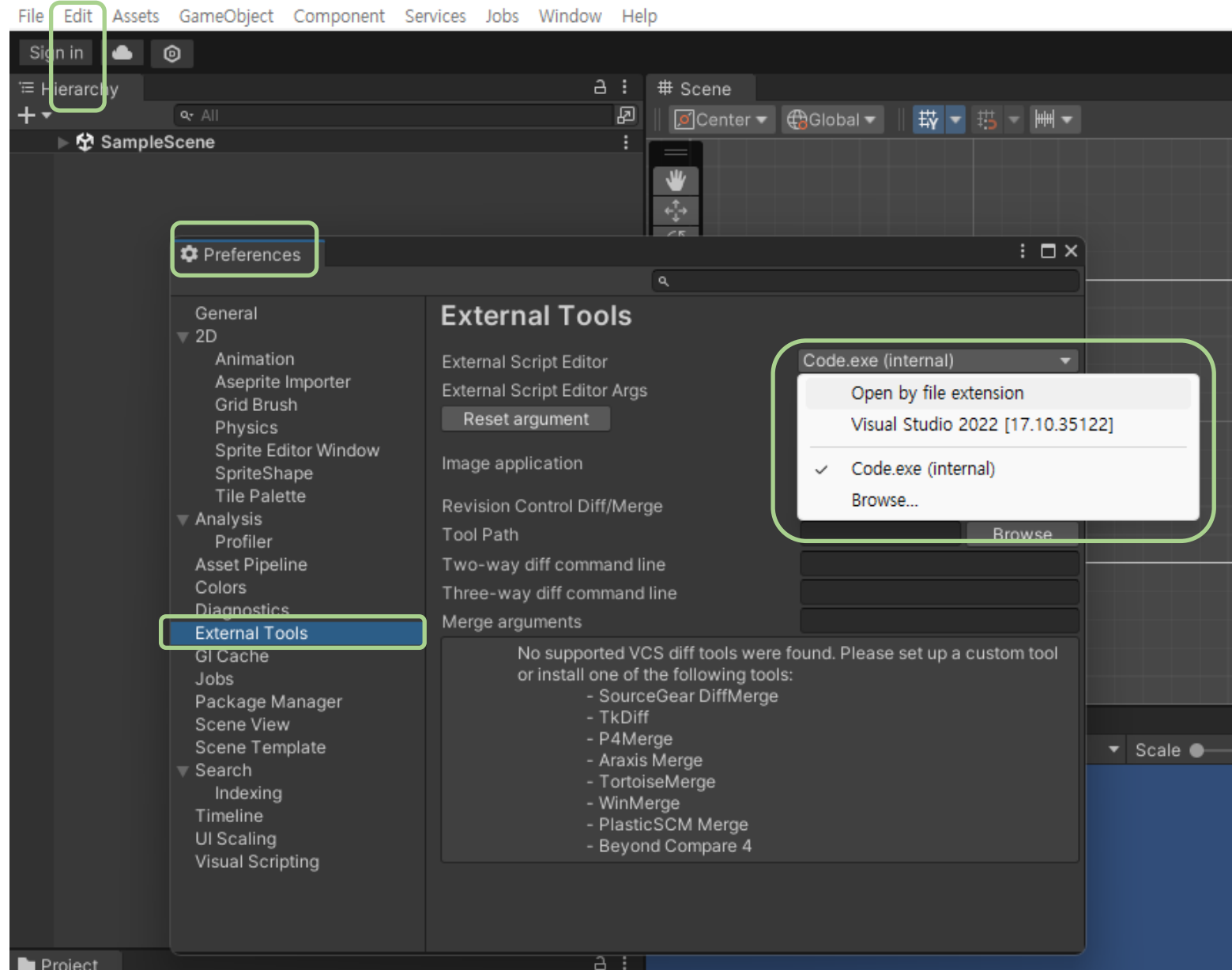




Unity Preference

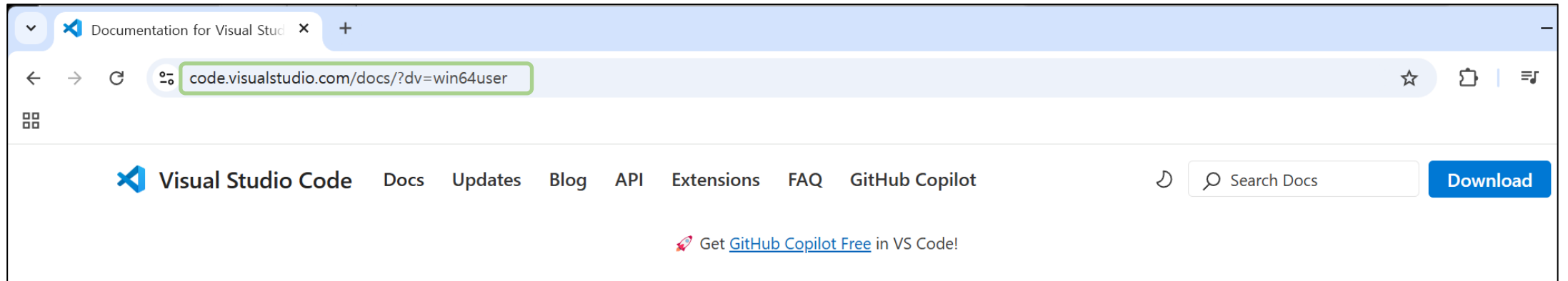


스크립트 편집 설정

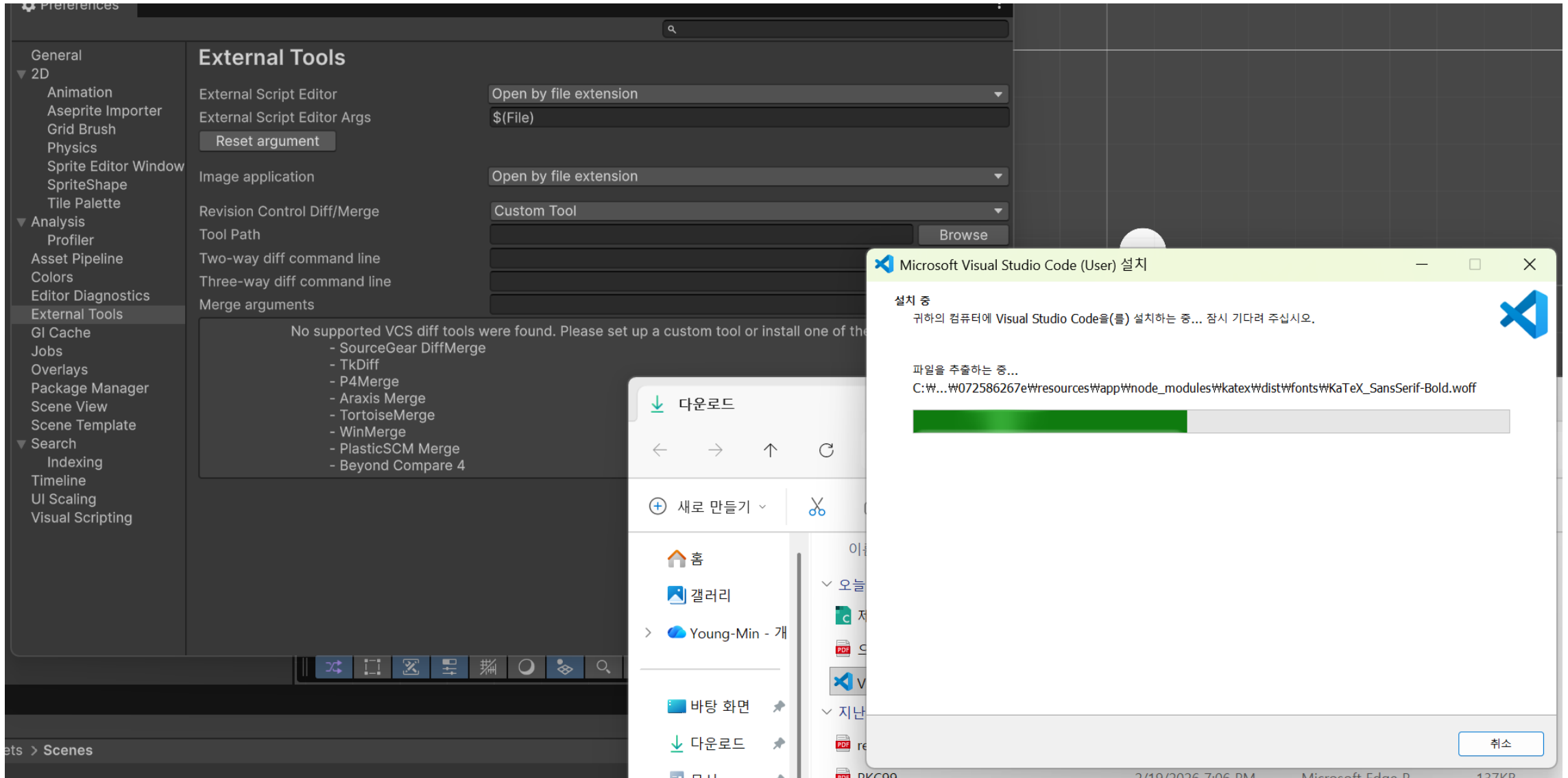


스크립트를 수정해 보자

- 스크립트를 두 번 클릭
 - 연결된 외부 개발 도구가 열림
 - Visual Studio Code를 연결해 두었음
 - 열리지 않는 경우에는 도구가 설치되지 않은 상태임
- 필요한 경우 설치



Microsoft Visual Studio Code 설치



필요한 Extension

The screenshot shows the Visual Studio Code interface with the C# Dev Kit extension installed. The left sidebar displays the 'EXTENSIONS: MARKETPLACE' view, where the 'C# Dev Kit' extension by Microsoft is highlighted with a green box. The main editor area shows the 'C# Dev Kit' extension details page, which includes the extension's icon, name, publisher (Microsoft), version (8,444,992), and a 4.5-star rating (157 reviews). The page also features buttons for 'Disable', 'Uninstall', and 'Switch to Pre-Release Version', along with an 'Auto Update' checkbox. The 'DETAILS' tab is selected, showing a description of the extension and a list of features.

C# Dev Kit
Microsoft [microsoft.com](#) | 8,444,992 | ★★★★★ (157)
Official C# extension from Microsoft

[Disable](#) [Uninstall](#) [Switch to Pre-Release Version](#) ☒ Auto Update [Settings](#)

[DETAILS](#) [FEATURES](#) [CHANGELOG](#) [DEPENDENCIES](#)

C# Dev Kit for Visual Studio Code

C# Dev Kit helps you manage your code with a solution explorer and test your code with integrated unit test discovery and execution, elevating your C# development experience wherever you like to develop (Windows, macOS, Linux, and even in a Codespace).

This extension builds on top of the great C# language capabilities provided by the [C# extension](#) and enhances your C# environment by adding a set of powerful tools and utilities that integrate natively with VS Code to help C# developers write, debug, and maintain their code faster and with fewer errors. Some of this new tooling includes but is not limited to:

- C# project and solution management via an integrated solution explorer
- Native testing environment to run and debug tests using the Test Explorer
- Roslyn-powered language service for best in-class C# language features such as code navigation, refactoring, semantic awareness, and more

Quick Start

1. Install C# Dev Kit (The [C# extension](#) and the [.NET Install Tool](#) will automatically be installed).
2. Open a folder/workspace that contains a C# project (.csproj) and the extension will activate, or use the ".NET: New Project..." command to create a new C# project.
3. Check out the [C# Getting Started](#) documentation to learn more!

Feature List & Walkthrough

- [Project System / Solution Explorer](#)
 - Solution Node Actions
 - Add Project
 - Build/Run Project

필요한 Extension

The screenshot displays the Visual Studio Code interface with the Unity extension marketplace open. The left sidebar shows a list of extensions, with 'Unity' highlighted. The main panel shows the details for the 'Unity' extension by Microsoft, which integrates Visual Studio Code with Unity. The extension is currently installed, and the 'Uninstall' button is visible. Below the extension details, there is a section titled 'Unity for Visual Studio Code' which describes the extension's purpose and lists its features.

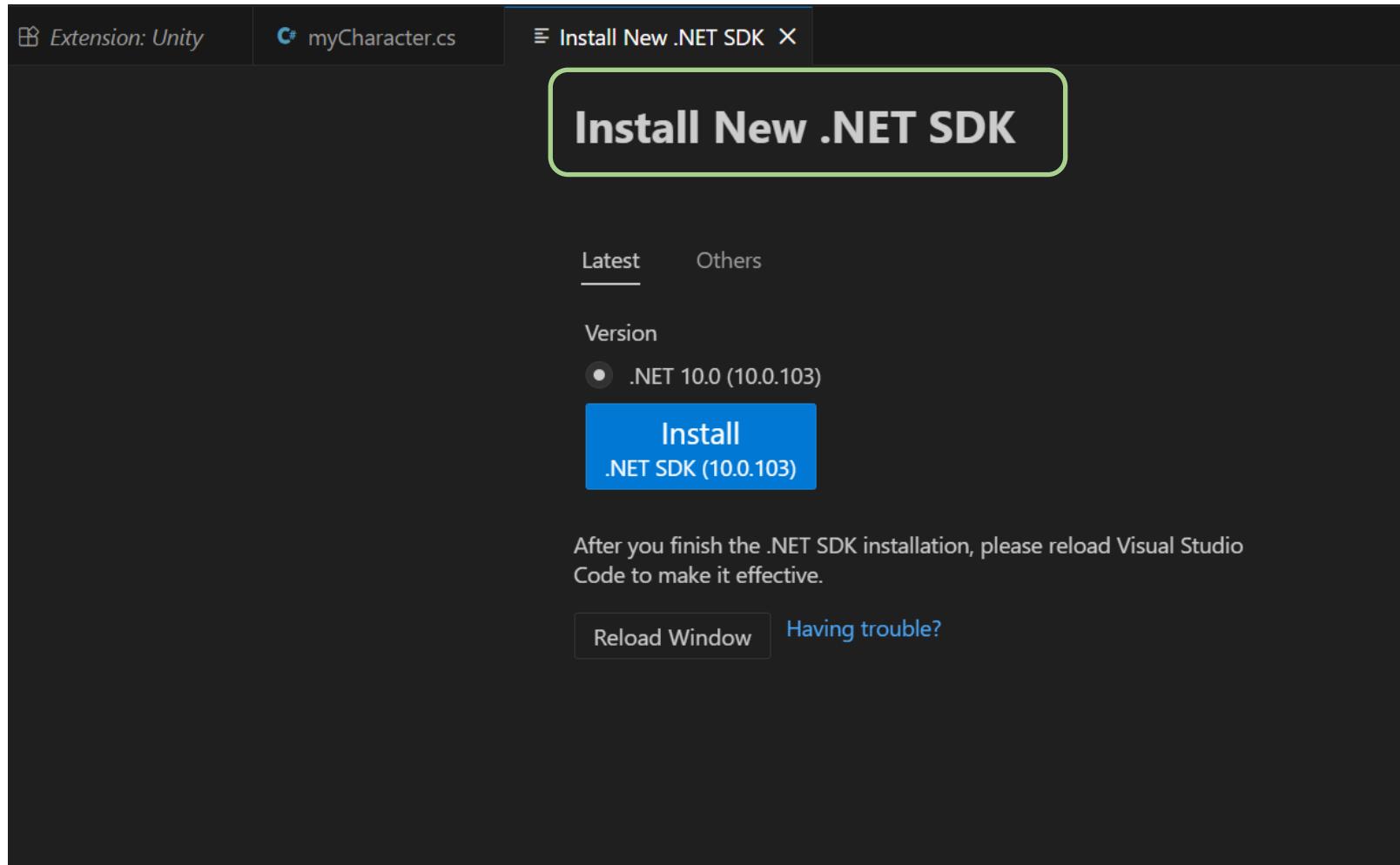
Unity for Visual Studio Code

The Unity extension provides C# developers with a lightweight and streamlined [Unity](#) development experience on Visual Studio Code. It builds on top of the rich C# capabilities provided by the [C# Dev Kit](#) and [C#](#) extensions. This extension integrates natively with Visual Studio Code and includes multiple productivity features including:

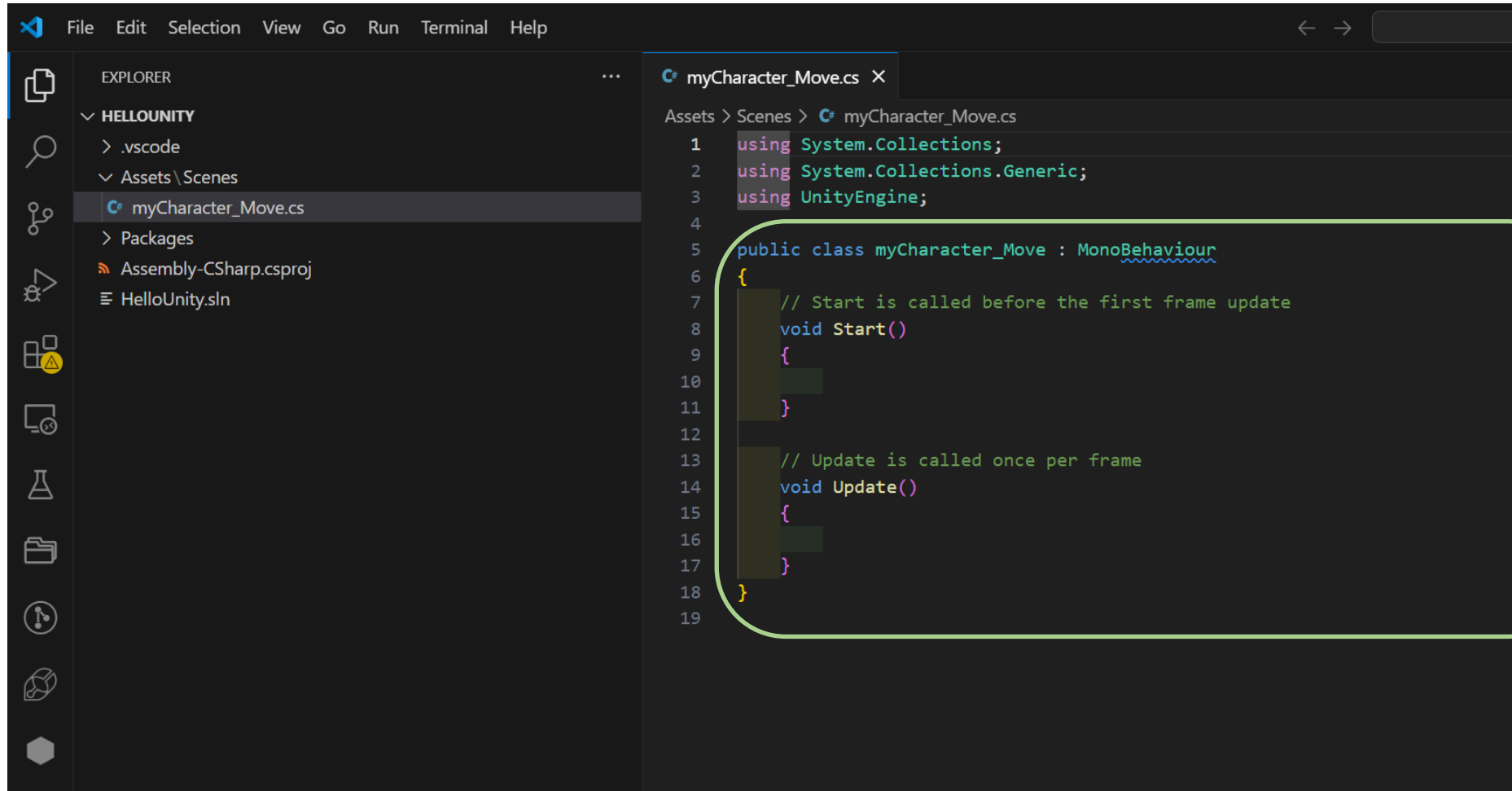
- A Unity debugger to debug your Unity editor and your Unity players.
- Unity specific C# analyzers and refactorings.
- Code coloration for Unity file formats (.asmdef, .shader, .uss, .uxml).

The background image shows the Unity extension marketplace in Visual Studio Code. The 'Unity' extension by Microsoft is highlighted with a green box. Below it, other extensions like 'Debugger for Unity', 'Unity Tools', 'Unity Code Snippets', 'Unity Snippets', 'eppzi! (C# theme for Unity)', 'Unity Toolbox', 'Unity Snippets Modified', 'Unity Code Snippets', 'Unity Dev Pack', and 'Unity Code' are listed. The right side of the image shows the details page for the 'Unity' extension, which includes the Unity logo, the extension name, the publisher 'Microsoft', and the description 'Integrates Visual Studio Code with Unity'. It also shows the number of downloads (1,276,272) and the average rating (4.5 stars from 75 reviews). Below this, there are buttons for 'Disable', 'Uninstall', and 'Auto Update'. The bottom section of the image shows a preview of the Unity extension in action, displaying a 3D game scene and a C# script in the Unity Inspector.

필요한 Extension



코드 편집

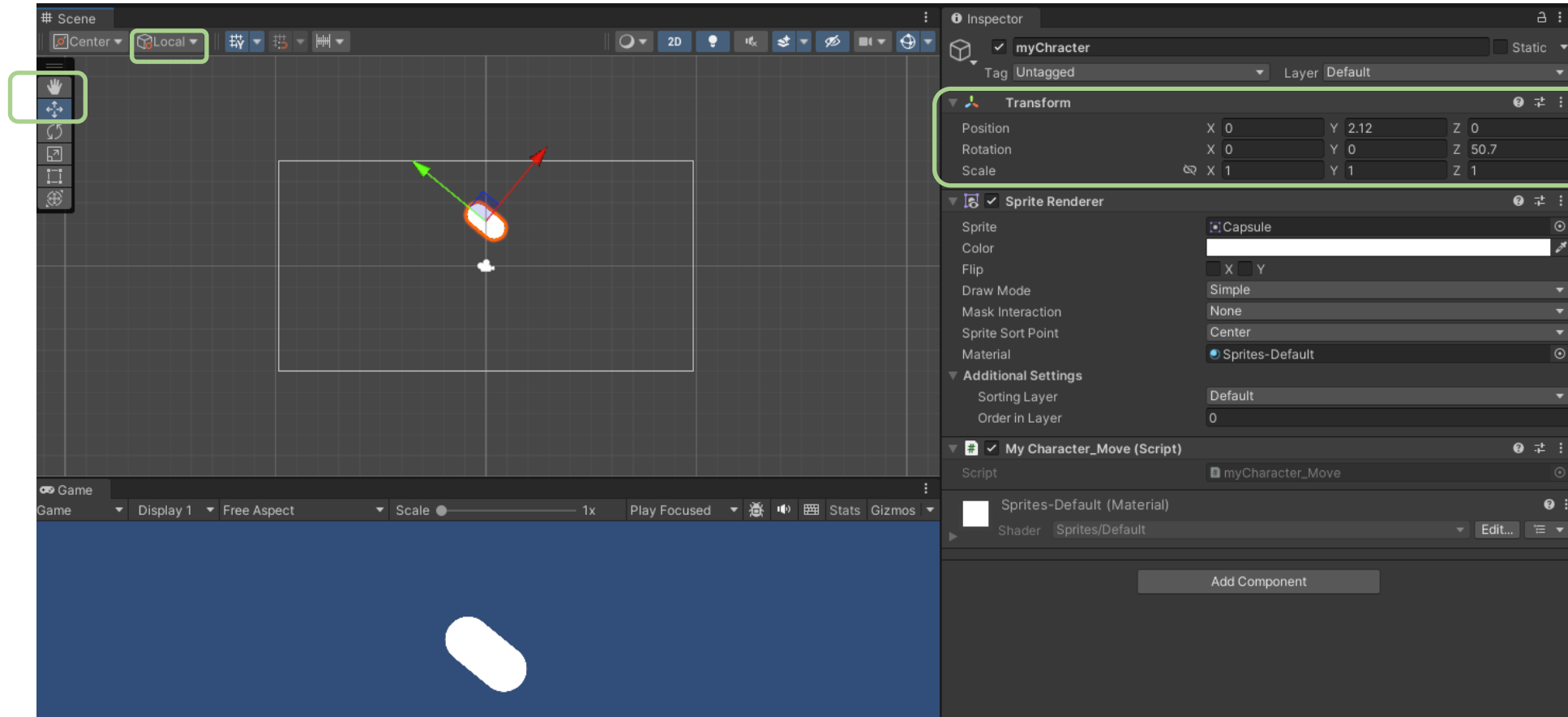


이 클래스가
우리의 캐릭터에
연결되어 있음

두 개의 클래스
Start, Update는
자동으로 호출되는
메소드

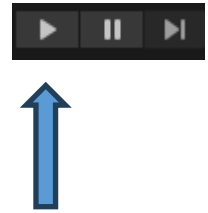
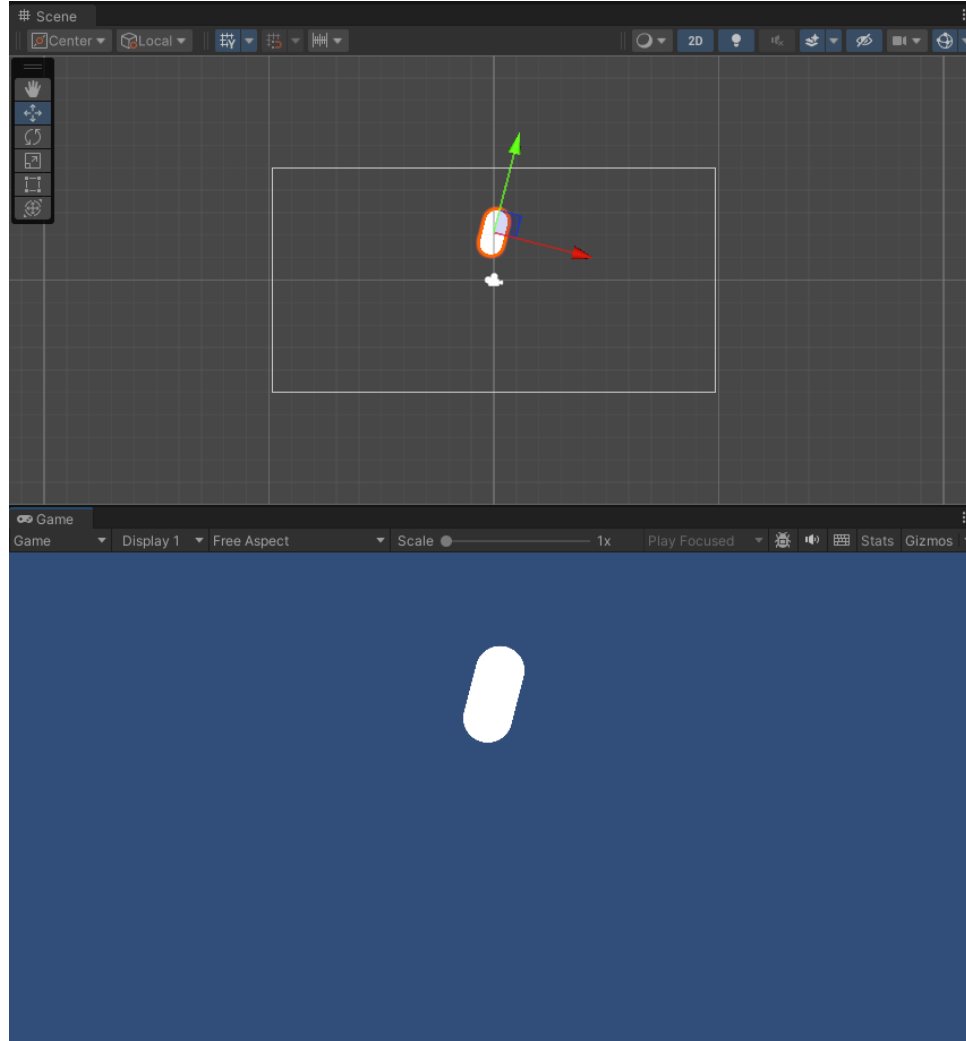
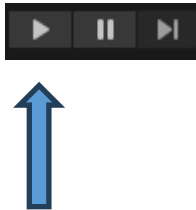
Start: 플레이될 때
Update: 매 프레임

인스펙터(Inspector)로 값의 변화 확인



움직이는 첫 프로젝트

Play 버튼을 누른다



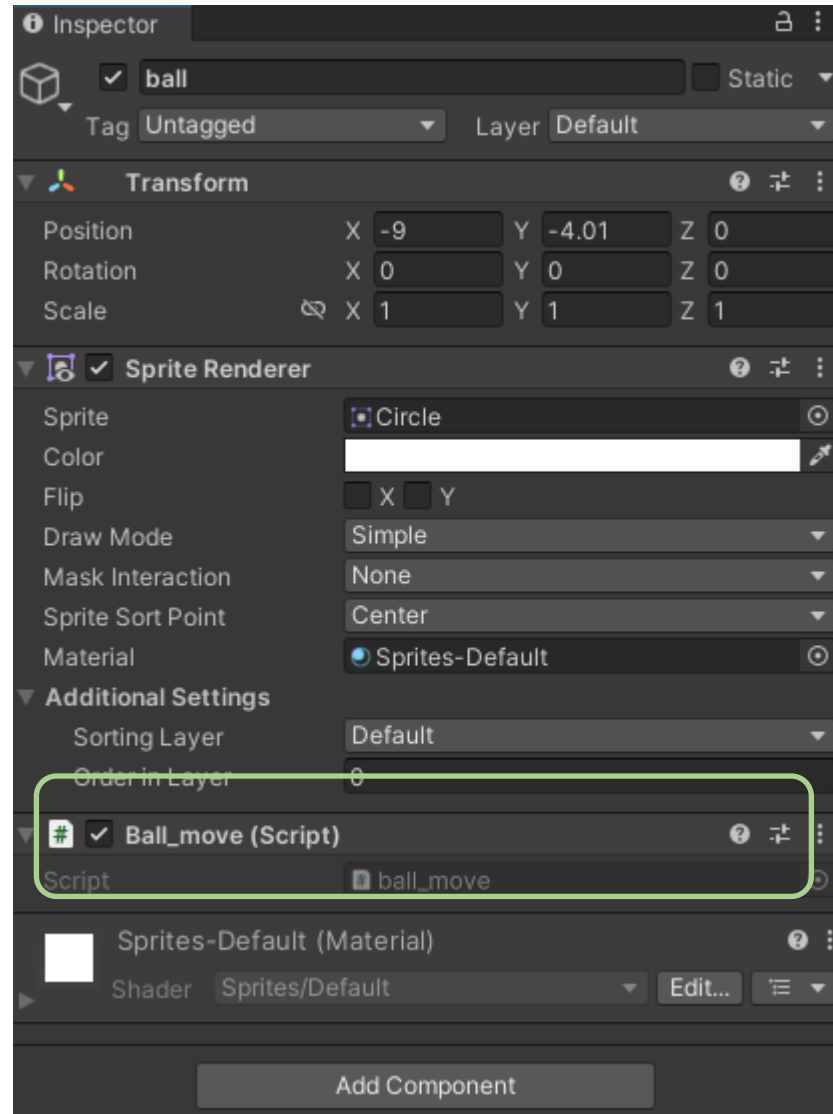
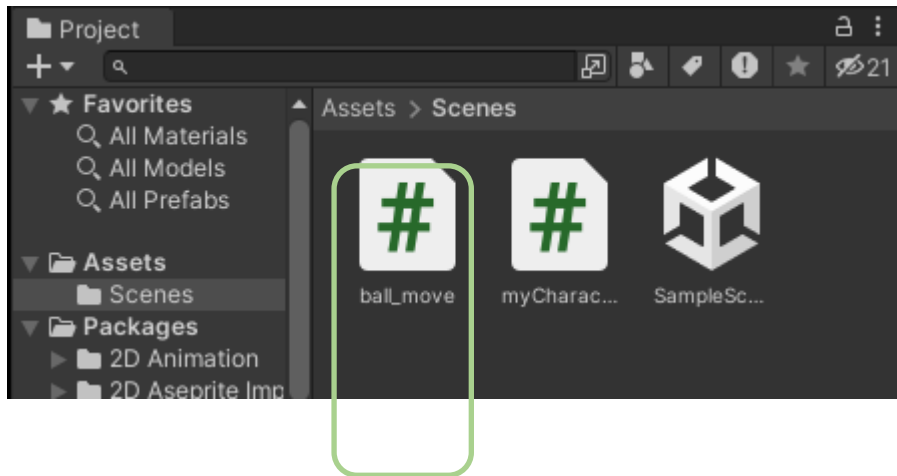
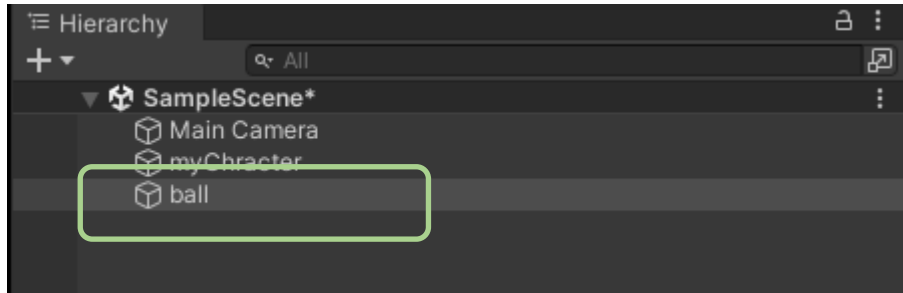
멈춰야지 변경이 가능

새로운 메소드: transform.Translate

```
13 // Update is called once per frame
    0 references
14 void Update()
15 {
16     transform.Rotate(0, 0.0f, 0.5f);
17     transform.Translate(0.0f, 0.01f, 0.0f);
18 }
19 }
20
```



새로운 캐릭터 만들어 보기



변수를 써서 코딩해 보기

```
using UnityEngine;
```

```
public class ball_move : MonoBehaviour  
{
```

```
    float mass = 2.0f;  
    float x = -9.0f;  
    float y = -4.0f;  
    float vx = 0.0f;  
    float vy = 0.0f;  
    float fx = 0.0f;  
    float fy = 0.0f;  
    float g = 9.8f;  
    float total_time = 0.0f;
```

```
// Start is called before the first frame update
```

```
void Start()
```

```
{  
    fx = 40.0f;  
    fy = 40.0f;  
}
```

```
// Update is called once per frame
```

```
void Update()
```

```
{  
    float dt = Time.deltaTime;  
    total_time += dt;  
    if (total_time > 0.5f)  
    {  
        fx = 0.0f;  
        fy = 0.0f;  
    }  
    float ax = fx / mass;  
    float ay = fy / mass - g;  
    vx += ax * dt; // v = u + at  
    vy += ay * dt;  
    x += vx * dt; // s = s + vt  
    y += vy * dt;  
    if( y < -4.5f) {  
        vy = -vy;  
        y = -4.5f + (-4.5f - y);  
    }  
    if( x > 10.0f) {  
        vx = -vx;  
        x = 10.0f - (x - 10.0f);  
    }  
    if( x < -10.0f) {  
        vx = -vx;  
        x = -10.0f + (-10.0f - x);  
    }  
    transform.position = new Vector3(x, y, 0);  
}
```

C#의 특징

1. 객체지향 프로그래밍(OOP) 지원

- 클래스, 상속, 다형성, 캡슐화 지원
- 인터페이스 및 추상 클래스 제공
- 코드 재사용성과 유지보수성이 뛰어남

1 객체지향 프로그래밍(OOP)

✓ 클래스 사용

<> C#

```
public class ballBounce : MonoBehaviour
```

- `class` 키워드를 통해 객체를 정의
- 공의 동작(데이터 + 기능)을 하나의 객체로 캡슐화

✓ 상속

<> C#

```
: MonoBehaviour
```

- `MonoBehaviour` 를 상속
- Unity의 생명주기 함수(Start, Update 등)를 사용할 수 있음

👉 객체지향 특징:

데이터(변수)와 동작(Update, Start)을 하나의 객체 안에 묶어 관리

C#의 특징

2. 타입 안전성 (Type Safety)

- 강력한 정적 타입 언어
- 컴파일 시점에 오류를 많이 검출
- 가비지 컬렉션(GC)으로 메모리 관리 자동화

2 타입 안전성 (Type Safety)

✓ 명확한 타입 선언

<> C#

```
float mass = 2.0f;  
float vx = 0.0f;  
float g = 9.8f;
```



- 모든 변수는 `float` 타입으로 명확히 선언
- 다른 타입을 대입하면 컴파일 오류 발생

예:

<> C#

```
mass = "hello"; // 컴파일 에러
```



👉 C#은 강한 정적 타입 언어

C#의 특징

3. .NET 플랫폼 기반

- .NET의 CLR(Common Language Runtime) 위에서 실행
- 다양한 라이브러리 제공
- Windows, 웹, 모바일, 게임 등 다양한 환경에서 개발 가능

C#의 특징

4. 간결하고 현대적인 문법

- 자동 프로퍼티 (auto-property)
- LINQ(Language Integrated Query)
- async/await 비동기 프로그래밍 지원
- 람다식 및 익명 메서드 지원

4 간결하고 현대적인 문법

✓ 자동 형 계산 및 연산자 사용

⟨⟩ C#

```
vx += ax * dt;  
x += vx * dt;
```



- 복합 대입 연산자 사용 (+=)
- 수식 표현이 간결함

✓ 구조체 사용

⟨⟩ C#

```
new Vector3(x, y, 0);
```



- 객체 생성 문법 new
- 생성자 호출 방식이 간결

👉 C#은 수학적 계산을 자연스럽게 표현 가능

C#의 특징

5. 다양한 애플리케이션 개발 가능

- Windows 응용프로그램 (WinForms, WPF)
- 웹 개발 (ASP.NET)
- 모바일 앱 (Xamarin, .NET MAUI)
- 게임 개발 (Unity 엔진)

5 다양한 애플리케이션 개발 가능

이 코드는 게임 엔진인 Unity에서 사용됨.

즉 C#은:

- 콘솔 프로그램
- 웹 (ASP.NET)
- 모바일 앱
- 그리고 Unity 게임 개발

까지 가능함.

이 코드 자체가 게임 개발에 C#이 사용되는 대표 사례

C#의 특징

6. 컴포넌트 지향 프로그래밍 지원

- 속성(Attribute) 기반 프로그래밍
- 이벤트 및 델리게이트 지원

6 컴포넌트 지향 프로그래밍

Unity는 컴포넌트 기반 구조.

</> C#

```
public class ballBounce : MonoBehaviour
```

- 이 스크립트는 하나의 “컴포넌트”
- GameObject에 붙여서 동작

</> C#

```
transform.position = new Vector3(x, y, 0);
```

- `transform` 컴포넌트에 접근

👉 C#은 이런 컴포넌트 구조를 자연스럽게 지원



Questions?